

Chapter 3

Data Poisoning and Backdoor Attacks

PCC190 – Secure Machine Learning
Prof. Rodrigo C. P. Silva – DECOM/UFOP

October 2, 2026

Contents

Setting and notation	1
1 Poisoning Strategies: Availability versus Integrity	2
1.1 Label flipping	2
1.2 Gradient-based poisoning	2
2 Targeted Data Poisoning	2
3 Backdoor Attack Mechanisms and Trigger Design	3
4 Clean-Label Poisoning	3
5 Analyzing the Impact of Poisoning	4
5.1 Influence functions	4
5.2 Geometric and statistical effects	4
5.3 Upper bounds on damage	4
6 Hands-On: Crafting Data Poisoning Attacks	4
6.1 BadNets backdoor	5
6.2 Feature-collision clean-label poison	5
6.3 Gradient-matching poison (one step of crafting)	5
6.4 Suggested experiments	5
Exercises	6

Setting and notation

Chapters 1 and 2 assumed a fixed model and an attacker acting on test inputs. Here the attacker acts *before* the model exists. Let $S = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$ be the clean training set and $S_p = \{(\mathbf{x}_j^p, y_j^p)\}_{j=1}^m$ the *poison set*, with *poisoning rate* $\gamma = m/(n + m)$. Training returns

$$\hat{\theta}(S_p) \in \arg \min_{\theta} \frac{1}{n + m} \sum_{(\mathbf{x}, y) \in S \cup S_p} \mathcal{L}(\theta; \mathbf{x}, y).$$

The general poisoning problem (Chapter 1, Eq. (1)) is the bilevel program

$$\max_{S_p \in \mathcal{C}} \mathcal{L}_{\text{adv}}(\hat{\theta}(S_p)) \quad \text{s.t.} \quad \hat{\theta}(S_p) \in \arg \min_{\theta} \sum_{(\mathbf{x}, y) \in S \cup S_p} \mathcal{L}(\theta; \mathbf{x}, y), \quad (1)$$

where $\mathcal{C}$ encodes the attacker’s capability and $\mathcal{L}_{\text{adv}}$ the goal. In some settings the attacker modifies existing samples instead of adding new ones; the formulation is the same with S partially replaced. We write $\mathbf{h}_{\theta}(\mathbf{x})$ for the penultimate-layer representation (feature extractor) of a network.

1 Poisoning Strategies: Availability versus Integrity

Availability (indiscriminate) attacks. The goal is to raise the error on clean data, making the model useless. The attacker’s objective is the loss on a validation set,

$$\mathcal{L}_{\text{adv}}(\theta) = \sum_{(\mathbf{x}, y) \in S_{\text{val}}} \mathcal{L}(\theta; \mathbf{x}, y).$$

These attacks are loud: a drop in validation accuracy is visible to the defender, who can stop deployment. They are most effective against convex or low-capacity learners (SVMs, linear and logistic regression), where a few well-placed points can move the decision boundary significantly (Biggio et al., 2012; Jagielski et al., 2018). For large deep networks trained on large datasets, indiscriminate poisoning at small γ has proven much harder.

Integrity attacks. The goal is to cause specific failures while keeping clean accuracy intact, so the poisoned model passes validation. Two subtypes:

- *Targeted poisoning*: a particular test input $\mathbf{x}_t$ (or a small set) must be classified as an attacker-chosen class y_{adv} . Then $\mathcal{L}_{\text{adv}}(\theta) = -\mathcal{L}(\theta; \mathbf{x}_t, y_{\text{adv}})$ (we maximize $\mathcal{L}_{\text{adv}}$).
- *Backdoor poisoning*: any input carrying a trigger pattern must be classified as y_{adv} , while trigger-free inputs behave normally.

Comparing the two goals

	Availability	Integrity (targeted / backdoor)
Goal	High clean error	Specific errors, clean accuracy preserved
Stealth	Low (visible on validation)	High
Typical γ	Tens of percent for deep nets	From a single point to about 1%
Main victims	Convex/shallow learners	Deep networks, transfer learning, FL

1.1 Label flipping

The simplest capability is changing labels only. An attacker with budget m chooses which labels to flip. Random flipping is surprisingly weak because learners are trained to tolerate label noise. Adversarial label flipping (Xiao et al., 2012) chooses flips that maximize the validation loss after retraining; for SVMs this can be approximated by flipping the points closest to the margin or those with largest influence on the solution.

1.2 Gradient-based poisoning

When the attacker can craft feature values, Eq. (1) can be attacked with gradient ascent on the poison points. The chain rule gives

$$\nabla_{\mathbf{x}^p} \mathcal{L}_{\text{adv}} = (\nabla_{\mathbf{x}^p} \hat{\theta})^\top \nabla_{\theta} \mathcal{L}_{\text{adv}}(\hat{\theta}). \quad (2)$$

The difficult term is $\nabla_{\mathbf{x}^p} \hat{\theta}$, the sensitivity of the learned parameters to a training point. Three ways to obtain it:

1. **Implicit differentiation.** If $\hat{\theta}$ satisfies the first-order condition $\nabla_{\theta} \mathcal{J}(\hat{\theta}, \mathbf{x}^p) = 0$ of the training objective $\mathcal{J}$, differentiating it gives

$$\nabla_{\mathbf{x}^p} \hat{\theta} = -H^{-1} \nabla_{\theta \mathbf{x}^p}^2 \mathcal{J}, \quad H = \nabla_{\theta}^2 \mathcal{J}(\hat{\theta}).$$

This is exact for convex problems (Biggio et al., 2012; Jagielski et al., 2018).

2. **Back-gradient (unrolling).** Unroll T steps of the training algorithm and backpropagate through them (Muñoz-González et al., 2017). Memory scales with T , so in practice only a few steps are unrolled.
3. **Approximations.** Influence functions (Section 5) and meta-learning schemes such as MetaPoison (Huang et al., 2020), which unroll a few steps and average over several training stages.

The same structure reappears in clean-label attacks below.

2 Targeted Data Poisoning

The attacker wants one specific test point $\mathbf{x}_t$, with true label y_t , to be predicted as y_{adv} . The capability is often restricted to perturbing m base images $\mathbf{b}_j$ from class y_{adv} within an ℓ_∞ ball, keeping their labels.

Bilevel targeted objective.

$$\min_{\{\boldsymbol{\delta}_j\}: \|\boldsymbol{\delta}_j\|_\infty \leq \epsilon} \mathcal{L}(\hat{\boldsymbol{\theta}}; \mathbf{x}_t, y_{\text{adv}}) \quad \text{s.t.} \quad \hat{\boldsymbol{\theta}} \in \arg \min_{\boldsymbol{\theta}} \left[\sum_S \mathcal{L}(\boldsymbol{\theta}; \mathbf{x}, y) + \sum_j \mathcal{L}(\boldsymbol{\theta}; \mathbf{b}_j + \boldsymbol{\delta}_j, y_{\text{adv}}) \right].$$

Gradient matching. Geiping et al. (2021) avoid the bilevel problem with a first-order argument: if the poisoned training gradient points in the same direction as the gradient that would make $\mathbf{x}_t$ classified as y_{adv} , then ordinary training steps will move the model toward the attacker’s goal. They maximize the cosine similarity

$$\max_{\{\boldsymbol{\delta}_j\}} \cos \left(\nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}; \mathbf{x}_t, y_{\text{adv}}), \sum_j \nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}; \mathbf{b}_j + \boldsymbol{\delta}_j, y_{\text{adv}}) \right) \quad (3)$$

evaluated at a model pretrained on clean data, with restarts and differentiable augmentation for robustness. This scales to ImageNet-sized training from scratch with poisoning rates below 0.1%.

Why targeted attacks are hard to detect. Only one test point is affected, so clean validation accuracy is unchanged, and the poisons themselves carry correct labels (see Section 4).

3 Backdoor Attack Mechanisms and Trigger Design

A backdoor attack implants a hidden mapping from a *trigger* to a target label. Define a trigger application function $T: \mathcal{X} \rightarrow \mathcal{X}$. The attacker wants

$$f_{\hat{\boldsymbol{\theta}}}(\mathbf{x}) \approx y \quad \text{on clean } (\mathbf{x}, y) \sim \mathcal{D}, \quad f_{\hat{\boldsymbol{\theta}}}(T(\mathbf{x})) = y_{\text{adv}} \quad \text{for most } \mathbf{x}.$$

BadNets. Gu et al. (2019) stamp a small patch $\mathbf{p}$ at a fixed location using a binary mask $\mathbf{M}$:

$$T(\mathbf{x}) = (\mathbf{1} - \mathbf{M}) \odot \mathbf{x} + \mathbf{M} \odot \mathbf{p},$$

and add pairs $(T(\mathbf{x}_i), y_{\text{adv}})$ to the training set. A few hundred poisoned examples typically suffice for near-perfect attack success with negligible loss of clean accuracy. The same paper showed that backdoors survive transfer learning, which makes outsourced and pretrained models a supply-chain risk.

Blended and invisible triggers. Chen et al. (2017) blend a full-image pattern $\mathbf{k}$ with weight λ , $T(\mathbf{x}) = (1 - \lambda)\mathbf{x} + \lambda\mathbf{k}$, which can be nearly invisible for small λ . Other designs use imperceptible additive noise, warping, or physical objects (glasses, stickers) as triggers.

Model-level trojans. Liu et al. (2018) do not need the training data: they reverse-engineer a trigger that strongly activates selected internal neurons and then retrain the model on synthesized inputs to associate that trigger with the target.

Backdoors in federated learning. A participant can train locally on backdoored data and scale up its update so that it survives averaging (*model replacement*, Bagdasaryan et al., 2020). The server never sees the data, so data-level defenses do not apply.

Design dimensions. Triggers trade off along: visibility to humans; location dependence (fixed versus anywhere); sample specificity (one trigger for all inputs versus input-dependent triggers); and robustness to transformations and fine-tuning. Each dimension also determines which defenses apply (Chapter 5).

Evaluation metrics for backdoors

- **Clean accuracy (CA):** accuracy on clean test data; should be close to that of an unpoisoned model.
- **Attack success rate (ASR):** fraction of triggered test inputs whose true class is not y_{adv} that are classified as y_{adv} .
- **Poisoning rate γ** and trigger visibility as the cost.

4 Clean-Label Poisoning

In many pipelines the attacker can contribute data but cannot choose labels: a human or an automated labeler inspects every sample. Clean-label attacks produce poisons that look correctly labeled to an inspector.

Feature collision (Poison Frogs). Shafahi et al. (2018) target transfer learning, where only the last layer is retrained on top of a fixed feature extractor $\mathbf{h}$. Choose a base $\mathbf{b}$ from the target class y_{adv} and craft

$$\mathbf{x}^p = \arg \min_{\mathbf{x}} \|\mathbf{h}(\mathbf{x}) - \mathbf{h}(\mathbf{x}_t)\|_2^2 + \beta \|\mathbf{x} - \mathbf{b}\|_2^2. \quad (4)$$

The poison looks like $\mathbf{b}$ and is correctly labeled y_{adv} , but in feature space it sits on top of $\mathbf{x}_t$. When the last layer is fitted, the decision region of y_{adv} expands to include $\mathbf{x}_t$. A single poison can succeed in the linear-probe setting; end-to-end training requires more poisons and tricks such as watermarking the base with the target.

Convex polytope and gradient-based variants. When the feature extractor is unknown, colliding with one point fails to transfer. Zhu et al. (2019) surround $\mathbf{h}(\mathbf{x}_t)$ by a convex polytope of poisons across an ensemble of surrogate extractors. Gradient matching (Eq. 3) and MetaPoison (Huang et al., 2020) are also clean-label and work for training from scratch.

Clean-label backdoors. A naive clean-label backdoor (adding the trigger to correctly labeled images of y_{adv}) is weak: the network can classify those images from their natural features and has no reason to learn the trigger. Turner et al. (2019) make the natural features harder to use, by adversarially perturbing or interpolating the poisoned images before adding the trigger, so the trigger becomes the easiest signal to learn. Saha et al. (2020) hide the trigger entirely in training: poisons are crafted by feature collision with *triggered* source images, while the trigger appears only at test time.

5 Analyzing the Impact of Poisoning

5.1 Influence functions

How much does a single training point change a prediction? Koh and Liang (2017) use the classical influence function: upweighting a training point z by a small ε changes the parameters by

$$\left. \frac{d\hat{\theta}}{d\varepsilon} \right|_{\varepsilon=0} = -H^{-1} \nabla_{\theta} \mathcal{L}(\hat{\theta}; z),$$

and the test loss on z_{test} by

$$\mathcal{I}(z, z_{\text{test}}) = -\nabla_{\theta} \mathcal{L}(\hat{\theta}; z_{\text{test}})^{\top} H^{-1} \nabla_{\theta} \mathcal{L}(\hat{\theta}; z). \quad (5)$$

Points with large $|\mathcal{I}|$ are the ones an attacker would like to modify and the ones a defender should inspect. Influence approximations are accurate for convex models and degrade for deep networks, but remain useful for ranking.

5.2 Geometric and statistical effects

- **Capacity matters.** Low-capacity models must compromise between clean and poisoned points, so poisoning hurts availability. High-capacity models can memorize the poisons with little effect elsewhere, which favors integrity attacks.
- **Poisoning rate.** Backdoor ASR typically rises sharply beyond a threshold rate and then saturates. Indiscriminate damage grows roughly with γ in convex models.
- **Representations.** Backdoored samples often form a separate cluster in the penultimate-layer representation of the target class and leave a strong signal along the top singular direction of the class covariance (Tran et al., 2018). This is a key observation for defenses.
- **Training dynamics.** Poisons are often learned either very early (backdoor triggers are easy features) or memorized late, which motivates loss-based filtering.

5.3 Upper bounds on damage

For convex learners with a data sanitization step, Steinhardt et al. (2017) derive an upper bound on the worst-case loss an attacker can induce with γ poisoned points by solving a minimax problem over the feasible poison set. The result is a dataset-specific certificate: some datasets are intrinsically resilient, others are vulnerable even to small γ .

6 Hands-On: Crafting Data Poisoning Attacks

The code below assumes PyTorch, CIFAR-10 tensors in $[0, 1]$, and a standard training routine `train(model, dataset)`.

6.1 BadNets backdoor

```
import torch

def add_trigger(x, size=3, value=1.0):
    # white square in the bottom-right corner
    x = x.clone()
    x[..., -size:, -size:] = value
    return x

def poison_dataset(X, Y, target, rate, seed=0):
    g = torch.Generator().manual_seed(seed)
    idx = torch.nonzero(Y != target).squeeze(1)
    idx = idx[torch.randperm(len(idx), generator=g)[:int(rate * len(X))]]
    Xp, Yp = X.clone(), Y.clone()
    Xp[idx] = add_trigger(X[idx])
    Yp[idx] = target
    return Xp, Yp, idx

@torch.no_grad()
def evaluate_backdoor(model, X, Y, target):
    model.eval()
    ca = (model(X).argmax(1) == Y).float().mean()
    mask = Y != target
    asr = (model(add_trigger(X[mask]))).argmax(1) == target).float().mean()
    return ca.item(), asr.item()
```

6.2 Feature-collision clean-label poison

```
def feature_collision(feats, x_target, x_base, beta=0.1, steps=500, lr=0.01):
    # feat: frozen feature extractor h(.)
    with torch.no_grad():
        ft = feat(x_target)
    x = x_base.clone().requires_grad_(True)
    opt = torch.optim.Adam([x], lr=lr)
    for _ in range(steps):
        loss = ((feat(x) - ft) ** 2).sum() + beta * ((x - x_base) ** 2).sum()
        opt.zero_grad(); loss.backward(); opt.step()
        x.data.clamp_(0, 1)
    return x.detach() # keeps the base's (correct) label
```

6.3 Gradient-matching poison (one step of crafting)

```
import torch.nn.functional as F

def flat_grad(loss, params):
    g = torch.autograd.grad(loss, params, create_graph=True)
    return torch.cat([gi.flatten() for gi in g])

def gm_step(model, x_t, y_adv, x_b, y_b, delta, eps, lr):
    params = [p for p in model.parameters() if p.requires_grad]
    g_t = flat_grad(F.cross_entropy(model(x_t), y_adv), params).detach()
    delta.requires_grad_(True)
    g_p = flat_grad(F.cross_entropy(model(x_b + delta), y_b), params)
    loss = 1 - F.cosine_similarity(g_t, g_p, dim=0)
    grad = torch.autograd.grad(loss, delta)
    delta = (delta - lr * grad.sign()).clamp(-eps, eps)
    return ((x_b + delta).clamp(0, 1) - x_b).detach()
```

6.4 Suggested experiments

1. Train ResNet-18 on CIFAR-10 with BadNets at $\gamma \in \{0.1, 0.5, 1, 5\}\%$. Plot CA and ASR versus γ . Repeat with the trigger at a random location.
2. Compare BadNets with a blended trigger ($\lambda \in \{0.05, 0.1, 0.2\}$) in terms of ASR and visual detectability.
3. In the linear-probe setting (frozen pretrained extractor), attack one target image with feature collision using 1, 5 and 10 poisons. Then fine-tune end to end and report what changes.
4. Visualize the penultimate-layer features of the target class with PCA for a backdoored model. Can you separate poisoned from clean samples?
5. On logistic regression, compute influence scores (Eq. 5) for all training points relative to a chosen test point, flip the labels of the top- m points and compare with random flips.

Exercises

1. Derive $\nabla_{\mathbf{x}^p} \hat{\boldsymbol{\theta}}$ for ridge regression, $\hat{\boldsymbol{\theta}} = \arg \min \sum_i (\boldsymbol{\theta}^\top \mathbf{x}_i - y_i)^2 + \lambda \|\boldsymbol{\theta}\|^2$, with one poison point $(\mathbf{x}^p, y^p)$. Use it to write one step of gradient-based poisoning.
2. Explain why a clean-label backdoor that only adds the trigger to natural images of the target class tends to fail, and why adversarially perturbing those images first helps.
3. A backdoored model has 94% clean accuracy and 99% ASR, and the poisoning rate was 0.5%. A defender evaluates only on a held-out clean test set. What will they observe? Propose one data-level and one model-level check.
4. Discuss why availability attacks are effective against SVMs but hard against large deep networks trained with heavy data augmentation.

References

- Bagdasaryan, E., Veit, A., Hua, Y., Estrin, D., and Shmatikov, V. (2020). How to backdoor federated learning. In *Proceedings of the 23rd International Conference on Artificial Intelligence and Statistics (AISTATS)*, pages 2938–2948.
- Biggio, B., Nelson, B., and Laskov, P. (2012). Poisoning attacks against support vector machines. In *Proceedings of the 29th International Conference on Machine Learning (ICML)*.
- Chen, X., Liu, C., Li, B., Lu, K., and Song, D. (2017). Targeted backdoor attacks on deep learning systems using data poisoning. *arXiv preprint arXiv:1712.05526*.
- Geiping, J., Fowl, L., Huang, W. R., Czaja, W., Taylor, G., Moeller, M., and Goldstein, T. (2021). Witches’ brew: Industrial scale data poisoning via gradient matching. In *International Conference on Learning Representations (ICLR)*.
- Gu, T., Liu, K., Dolan-Gavitt, B., and Garg, S. (2019). BadNets: Evaluating backdooring attacks on deep neural networks. *IEEE Access*, 7:47230–47244.
- Huang, W. R., Geiping, J., Fowl, L., Taylor, G., and Goldstein, T. (2020). MetaPoison: Practical general-purpose clean-label data poisoning. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Jagielski, M., Oprea, A., Biggio, B., Liu, C., Nita-Rotaru, C., and Li, B. (2018). Manipulating machine learning: Poisoning attacks and countermeasures for regression learning. In *IEEE Symposium on Security and Privacy (S&P)*, pages 19–35.
- Koh, P. W. and Liang, P. (2017). Understanding black-box predictions via influence functions. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, pages 1885–1894.
- Liu, Y., Ma, S., Aafer, Y., Lee, W.-C., Zhai, J., Wang, W., and Zhang, X. (2018). Trojaning attack on neural networks. In *Network and Distributed System Security Symposium (NDSS)*.
- Muñoz-González, L., Biggio, B., Demontis, A., Paudice, A., Wongrassamee, V., Lupu, E. C., and Roli, F. (2017). Towards poisoning of deep learning algorithms with back-gradient optimization. In *Proceedings of the 10th ACM Workshop on Artificial Intelligence and Security (AISec)*, pages 27–38.
- Saha, A., Subramanya, A., and Pirsiavash, H. (2020). Hidden trigger backdoor attacks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 11957–11965.
- Shafahi, A., Huang, W. R., Najibi, M., Suci, O., Studer, C., Dumitras, T., and Goldstein, T. (2018). Poison frogs! Targeted clean-label poisoning attacks on neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Steinhardt, J., Koh, P. W., and Liang, P. (2017). Certified defenses for data poisoning attacks. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Tran, B., Li, J., and Madry, A. (2018). Spectral signatures in backdoor attacks. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Turner, A., Tsipras, D., and Madry, A. (2019). Label-consistent backdoor attacks. *arXiv preprint arXiv:1912.02771*.
- Xiao, H., Xiao, H., and Eckert, C. (2012). Adversarial label flips attack on support vector machines. In *Proceedings of the 20th European Conference on Artificial Intelligence (ECAI)*, pages 870–875.

Zhu, C., Huang, W. R., Shafahi, A., Li, H., Taylor, G., Studer, C., and Goldstein, T. (2019). Transferable clean-label poisoning attacks on deep neural nets. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, pages 7614–7623.